Assignment 2: Designing methods for complex data

Goals: Learn to design methods for complex class hierarchies. Practice designing the representation of complex data.

Instructions

This homework should be done with your lab partner.

Be very, very careful with naming! Again, the solution files expect your submissions to be named a certain way, so that they can define their own `Examples` class with which to test your code. Therefore, whenever the assignment specifies:

- the names of classes,
- the names and types of the fields within classes,
- the names, types and order of the arguments to the constructor,
- the names, types and order of arguments to methods, or
- filenames,

...be sure that your submission uses exactly those names. Additionally, make sure you follow the style guidelines that the handin server enforces. For now the most important ones are: using spaces instead of tabs, indenting by 2 characters, following the naming conventions (data type names start with a capital letter, names of fields and methods start with a lower case letter), and having spaces before curly braces.

You will submit this assignment by the deadlines using the course handin server. Follow A Complete Guide to the Handin Server for information on how to use the handin server. **You may submit as many times as you wish. Be aware of the fact that close to the deadline the server may slow down to handle many submissions, so try to finish early.** There will be a separate submission for each problem - it makes it easier to grade each problem, and to provide you with the feedback for each problem you work on.

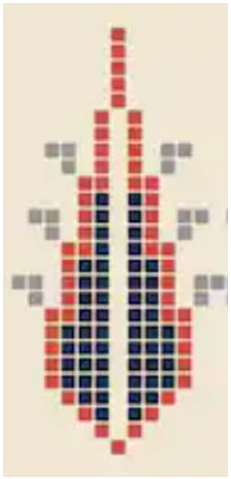
Practice Problems

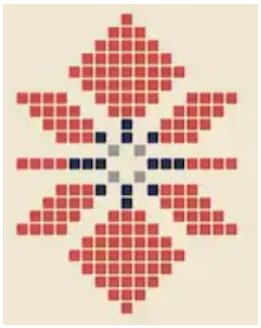
Work out these problems on your own. Save them in an electronic portfolio, so you can show them to your instructor, review them before the exam, use them as a reference when working on the homework assignments.

- Problem 10.6 on page 102
- Problem 11.2 on page 113
- Problem 12.1 on page 125
- Problem 14.7 on page 140
- Problem 15.2 on page 149
- Problem 15.3 on page 149
- Problem 15.8 on page 171

Problem 1: Embroidery

The art of creating decorative designs on fabric using a needle and thread is called embroidery. For instance, the following designs for a tree motif, a flower motif or bird motif may be created by an embroidery stitch such as a cross stitch or a chain stitch:







The following DrRacket data definition describes a simple piece of embroidery:

```
;;An EmbroideryPiece is a (make-embroidery-piece String Motif)
(define-struct embroidery-piece (name motif))

;; Motif is one of
;; -- CrossStitchMotif
;; -- ChainStitchMotif
;; -- GroupMotif

;; A CrossStitchMotif is a (make-cross-stitch-motif String Double)
(define-struct cross-stitch-motif (description difficulty))
;; interpretation: difficulty is a number between 0 and 5, with 5 being the most difficult

;; A ChainStitchMotif is a (make-chain-stitch-motif String Double)
(define-struct chain-stitch-motif (description difficulty))
;; interpretation: difficulty is a number between 0 and 5, with 5 being the most difficult

;; A GroupMotif is a (make-group-motif String [List-of Motif])
(define-struct group-motif (description motifs))
```

We are giving you the names of the classes or interfaces you will probably need — make sure you use these names to define your interfaces and classes.

A reminder on naming conventions: For lists of data, the names of the interface should always start with `ILo`, while the two classes' names start with `MtLo` for the empty lists and `ConsLo` for the nonempty lists; all three of these names should be followed by the name of the datatype of the elements of the list. So we would have `ILoString`, `MtLoString`, `ConsLoString` to represent lists of `String`s, `ILoBook`, `MtLoBook`, `ConsLoBook` to represent lists of `Book`s, etc.

- Draw a class diagram for the classes that represent this data definition. (It is optional to submit your diagram. You can draw this on paper, or in ASCII art as a comment in your submitted file.)
- Define Java classes that represent the definitions above.
- **Name your examples class `ExamplesEmbroidery`**

In the `ExamplesEmbroidery` class design the example of the following:

- A piece named `"Pillow Cover"`, with the following motifs:
  - A `"nature"` group motif that has a
    - `"bird"` cross-stitch motif with `4.5` difficulty, a
    - `"tree"` chain-stitch motif with `3.0` difficulty, and a
    - `"flowers"` group motif with a
      - `"rose"` cross-stitch motif with `5.0` difficulty, a
      - `"poppy"` chain-stitch motif with `4.75` difficulty, and a
      - `"daisy"` cross-stitch motif with `3.2` difficulty

Name this example `pillowCover` Our test program will check that the field `pillowCover` in the class `ExamplesEmbroidery` represents this information. (You may name the other motifs, and all the items inside them, anything you like, though the names should be reasonably descriptive.)

- Design the method `averageDifficulty` that computes the average difficulty of all of the cross-stitch and chain-stitch motifs in an EmbroideryPiece.

  Hint: you will need some helper methods for this...
- *Tricky!* Design the method `embroideryInfo` that produces one `String` that has in it all names of cross-stitch and chain-stitch motifs in an EmbroideryPiece, their stitch types in parentheses, and each motif separated by comma and space. The `String` should end in a period.

  So for the Pillow Cover example above this `String` would be

  ```
  "Pillow Cover: bird (cross stitch), tree (chain stitch), rose (cross stitch), poppy (chain stitch), daisy (cross stitch)."
  ```

  Note: You can combine two `String`s with a `+` operator, or by invoking the method `concat`: assuming both `s1` and `s2` are `String`s, then both `s1 + s2` and `s1.concat(s2)` do the same thing as the BSL expression `(string-append s1 s2)`.

  Note: There is a comma and space between any two entries, but not after the last one.

Problem 2: A Picture is worth a thousand words

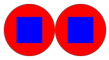
Define the file `Pictures.java` that will contain the entire solution to this problem.

For this problem, you're going to implement a small fragment of the image library you've been using for Fundies 1 and will be using in Fundies 2. Each *`picture`* is either a single `Shape` or a `Combo` that connects one or more pictures. Each `Shape` has a *`kind`*, which is a string describing what simple shape it is (e.g., `"circle"` or `"square"`), and a *`size`*. (For this problem, we will simplify and assume that each simple shape is as tall as it is wide.) A `Combo` consists of a *`name`* describing the resulting picture, and an *`operation`* describing how this image was put together.

There are three kinds of operations: `Scale` (takes a single *`picture`* and draws it twice as large), `Beside` (takes two pictures, and draws *`picture1`* to the left of *`picture2`*), and `Overlay` (takes two pictures, and draws *`top-picture`* on top of *`bottom-picture`*, with their centers aligned).

- Design the classes (and interfaces) needed to represent the given information.
- Sketch the class diagram for the classes and interfaces you have designed. (You can draw this on paper, or in ASCII art as a comment in your submitted file. You do not need to hand this in.)
- In the `ExamplesPicture` class define example data that represents the following images (the colors don't matter; they're just for illustration here):
  - A *`circle`* -  , a single circle of size 20
  - A *`square`* -  , a single square of size 30



- A *big circle* -  , the result of scaling `circle`
- A *square on circle* - 
- A *doubled square on circle* - 

  You should define each picture by its name (e.g. `square` or `bigCircle`). Any combo image should use the *description text* given above as its `description`.

  Our test program will use this data to test your methods.

- Add to your examples one more example of picture for each of the possible operations. Do not modify the existing pictures.
- Design the method `getWidth` that computes the overall width of this picture.

  Hint: follow the design recipe... working through examples really helps.
- Design the method `countShapes` that computes the number of single shapes involved in producing the final image.

  Note: Make sure you count every shape each time it is used.
- Design the method `comboDepth`, that computes how deeply operations are nested in the construction of this picture. For example, the `comboDepth` of the last example picture above is `3`.
- Design the method `mirror`. This should leave the entire image unchanged, except `Beside` combos, which should have their two sub-images flipped (all names can remain untouched). This mirroring should happen the entire way down the image.
- *Tricky!* Design the method `pictureRecipe` that takes an integer `depth`, and produces a `String` representing the contents of this picture, where the recipe for the picture has been expanded only `depth` times. For example, the `pictureRecipe` at depth 0 for the last example image above is `"doubled square on circle"`, at depth 2 is `"beside(overlay(square, big circle), overlay(square, big circle))"`, and at depth 3 or more is `"beside(overlay(square, scale(circle)), overlay(square, scale(circle)))"`.

  In more detail: invoking `pictureRecipe` on a `Combo` produces its name if the given `depth` is less than or equal to 0, and the formula of its mixture (at that depth) otherwise.

  Hint: If you get stuck, you may want to use a wish list to determine subproblems that may be of use to you, and that you can delegate to when needed.